Internationalization in Java

The Concepts

Haluk O. Bingol

Faculty of Computer and Information Sciences
Yeditepe University

v2026-10-15

Outline

1. Why Internationalization?
2. Locales
3. Characters and Encodings
4. Numbers, Money, Dates
5. Translating Messages
6. Sorting and Case
7. The Java Toolbox
8. Doing It Right
9. Summary

Why Internationalization?

The same numbers, five countries

Terminal codeoutput/SameValue.txt

```
$ java Show.java en-US tr-TR de-DE fr-FR de-CH
en-US  1,234,567.891  $1,234.50          25%   October 15, 2026
tr-TR  1.234.567,891  ₺1.234,50          %25   15 Ekim 2026
de-DE  1.234.567,891  1.234,50 €         25 %  15. Oktober 2026
fr-FR  1 234 567,891  1 234,50 €        25 %  15 octobre 2026
de-CH  1’234’567.891  CHF 1’234.50       25%   15. Oktober 2026
```

- The **data** is the same: 1234567.891, 1234.5, 0.25, 15 October 2026.
- The **presentation** differs: decimal point or comma, grouping with `,` `.` space or `’`, symbol before or after, %25 in Turkish, word order of dates.
- A program that hard-codes `"%.2f"` or `"MM/dd/yyyy"` is right in one country and wrong in the others.

Three words: i18n, l10n, g11n

i18n **internationalization**: design the program so that it *can* be adapted to any language and region *without changing the code*. Done once, by developers.

l10n **localization**: adapt it to *one* locale: translate texts, choose formats, pictures, colors. Done for every new market, often by translators.

g11n **globalization**: both together, plus the business side.

The numbers count the letters left out:
i nternationalizatio n has 18.

Rule of thumb

i18n is an **engineering** task: separate what varies from the code.

l10n is a **content** task: fill in the varying part for one place.

Cost

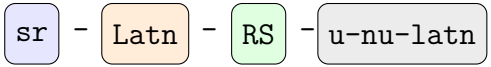
Adding i18n later means touching every string and every `format` call. Adding it from the start costs almost nothing.

What changes from one culture to another

Item	Example differences	Java
texts	“Save”, “Kaydet”, “Speichern”	`ResourceBundle`
messages	word order, plural forms	`MessageFormat`
numbers	`1,234.5` `1.234,5` `1 234,5`	`NumberFormat`
currency	`$12.50` `12,50 €` `₺12,50`	`NumberFormat`, `Currency`
dates, times	`10/15/26` `15.10.2026`, 12 or 24 hours	`DateTimeFormatter`
time zones	UTC+3 all year in Istanbul, daylight saving elsewhere	`ZoneId`, `ZonedDateTime`
calendars	Gregorian, Buddhist, Hijri, Japanese	`java.time.chrono`
sorting	where `ç`, `ı`, `ö` go in a list	`Collator`
upper/lower case	`i` → `İ` in Turkish, `ß` → `SS` in German	`toUpperCase(Locale)`
writing	alphabet, direction (left-to-right, right-to-left)	`Character`, `Bidi`
others	names, addresses, phone numbers, units, paper size	application code

Locales

A locale names a language and a region

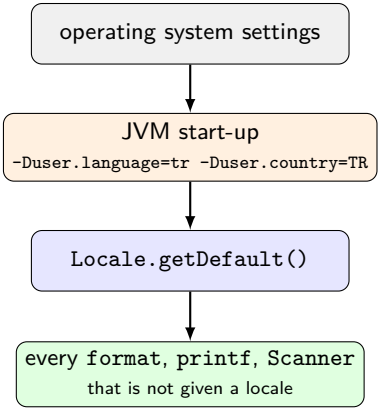


Serbian, written in the Latin alphabet, as used in Serbia. Only the language is required; the others are added when they matter.

tag	meaning
`tr`	Turkish
`tr-TR`	Turkish, Türkiye
`tr-CY`	Turkish, Cyprus
`en-US`	English, United States
`en-GB`	English, United Kingdom
`de-CH`	German, Switzerland
`pt-BR`	Portuguese, Brazil

Standard: **BCP 47** (IETF). Java writes `tr_TR` in `toString()` and `tr-TR` in `toLanguageTag()`.

Where does the default locale come from?



Windows

Settings » Time & language » Language & region:
Regional format

macOS

System Settings » General » Language & Region:
Region

The same program prints `3.14` on one laptop and `3,14` on the next. Programs that must give the same output everywhere pass a locale explicitly.

Characters and Encodings

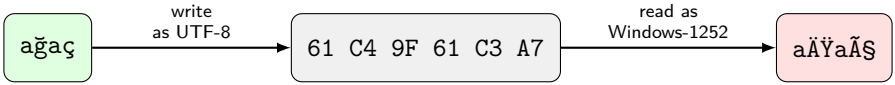
Unicode: one number for every character

- Every character has a **code point**: `A` = U+0041, `ğ` = U+011F, `ı` = U+0131, `€` = U+20AC, `₺` = U+20BA.
- About 150 000 characters: all modern scripts, symbols, emoji.
- An **encoding** turns code points into bytes. **UTF-8** is the standard today.
- Old 8-bit encodings had 256 characters: ISO-8859-9 and Windows-1254 for Turkish, ISO-8859-1 for Western Europe.

char	code point	UTF-8 bytes
`A`	U+0041	`41`
`ğ`	U+011F	`C4 9F`
`€`	U+20AC	`E2 82 AC`
a smiley emoji	U+1F600	`F0 9F 98 80`

UTF-8 uses 1 to 4 bytes; ASCII text stays the same.

Mojibake: the wrong encoding



- The bytes are fine; the **reader** guessed the wrong encoding. This garbage is called *mojibake*.
- Every place where text crosses a border needs an encoding: source files, `javac`, the console, files, databases, the network.
- Java 18 and later use **UTF-8** by default everywhere (JEP 400). Older Java used the operating system's encoding, e.g. Windows-1254 on a Turkish Windows.
- Inside the program a Java `String` is a sequence of 16-bit `chars` (UTF-16); the encoding only matters at the borders.

Numbers, Money, Dates

Numbers and money

locale	number	percent
en-US	1,234.5	25%
tr-TR	1.234,5	%25
de-DE	1.234,5	25 %
fr-FR	1 234,5	25 %
de-CH	1’234.5	25%

Parsing has the same problem: `"1.234"` is one thousand two hundred thirty-four in Türkiye and one point two three four in the USA.

Currency is two things

- the **currency**: TRY, EUR, USD, JPY (ISO 4217), with its own number of decimals (JPY has none);
- the **format**, which follows the user's locale: a German reads `12,50 $`, an American `$12.50`.

Never convert money with `double`; use `BigDecimal`.

Dates, times and time zones

Format

- order: `10/15/26` (US), `15.10.2026` (TR, DE), `2026-10-15` (ISO 8601)
- names: *Thursday*, *Perşembe*, *Donnerstag*
- clock: `2:30 PM` or `14:30`
- calendar: 2026 (Gregorian) is 2569 in the Thai Buddhist calendar

Store and exchange dates in **ISO 8601**; localize only for display.

Time zones

- the same moment: 14:30 in Istanbul is 13:30 in Berlin, 07:30 in New York, 20:30 in Tokyo (15 October 2026)
- zones have names like `Europe/Istanbul` (IANA database), not only offsets
- daylight saving time: clocks jump, some local times do not exist, others happen twice
- rules change by law; the JDK ships updates of the zone database

Translating Messages

Keep texts out of the code

Messages.properties

```
greet=Hello, {0}!
save=Save
```

Messages_tr.properties

```
greet=Merhaba, {0}!
save=Kaydet
```

Messages_de.properties

```
greet=Hallo, {0}!
save=Speichern
```

- The code uses **keys**: `greet`, `save`.
- One file per language: a **resource bundle**. A translator edits only these files.
- Placeholders `{0}`, `{1}` let each language put the values where its grammar wants them.
- Adding a language = adding a file, no new code.
- Missing a key in `_tr`? It is taken from the base file (*fallback*).

Finding the right file (simplified)

Algorithm 1: GetBundle($base, \ell$)

Input: base name b, requested locale ℓ (e.g. `tr-TR`), default locale δ
Output: a bundle whose parents form a chain down to b

```
C ← Candidates(b, ℓ)                    // Messages_tr_TR, Messages_tr
if none of C exists then
    C ← Candidates(b, δ)                // fall back to the default locale
append b to C                           // the base file is the last resort
return the first existing file of C, chained to the later ones
```

A key is looked up in the first file, then in its parents, so a translation may be incomplete.

Messages are more than words

Word order

- English: *Ali sent 3 files to Ayşe.*
- Turkish: *Ali, Ayşe'ye 3 dosya gönderdi.*

Never build sentences with +: `name + " sent " + n + " files"` cannot be translated. Use one pattern per language with numbered placeholders.

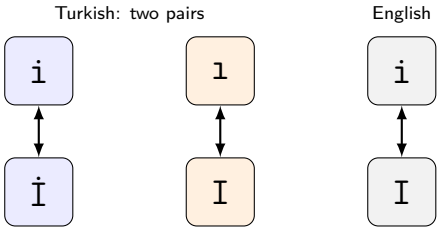
Plurals

language	forms after a number
English	1 file, 2 files
Turkish	1 dosya, 2 dosya (noun stays singular)
Polish	3 forms (1, 2–4, 5+)
Arabic	6 forms

The rules come from the Unicode CLDR project.

Sorting and Case

Upper and lower case depend on the language



`"title".toUpperCase()` gives `"TİTLE"` on a Turkish computer.

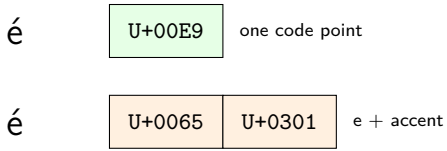
- Text for **people**: use the user's locale, `toUpperCase(locale)`.
- Text for **programs** (keywords, file names, HTTP headers, enum names): use `toUpperCase(Locale.ROOT)`.
- Famous bugs: `"FILE".toLowerCase().equals("file")` is **false** on Turkish systems; many libraries and tools failed this way.
- German ß becomes SS: the string gets longer.

Alphabetical order depends on the language

- `String.compareTo` compares **code point numbers**: `Z` (90) < `a` (97) < `ç` (231) < `ı` (305).
- People expect the **alphabet**: Turkish puts `ç` after `c`, `ı` before `i`.
- A **collator** knows the rules of one language and how strict to be (ignore case? ignore accents?).

by code point	Turkish alphabet
Ali	ağaç
araba	Ali
ağaç	araba
cam	cam
ilaç	çay
Ömer	ılık
çay	ilaç
ılık	Ömer

Same look, different characters: normalization



- Both look the same, but `equals` says **false**, and `length()` is 1 or 2.
- Text from keyboards, files and the web can come in either form (macOS file names often use the second).
- **Normalize** before comparing or storing: `java.text.Normalizer`, form NFC.

The Java Toolbox

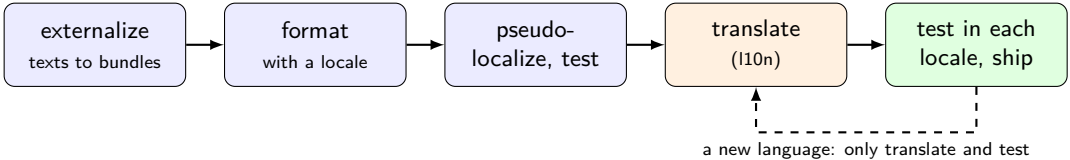
Where to find what

task	class	package
name a locale	`Locale`	`java.util`
translated texts	`ResourceBundle`	`java.util`
messages with values	`MessageFormat`, `ChoiceFormat`	`java.text`
numbers, money	`NumberFormat`, `Currency`, `BigDecimal`	`java.text`, `java.util`, `java.math`
dates and times	`DateTimeFormatter`, `ZonedDateTime`	`java.time`, `java.time.format`
sorting	`Collator`	`java.text`
normalization	`Normalizer`	`java.text`
words, sentences	`BreakIterator`	`java.text`
encodings	`Charset`, `StandardCharsets`	`java.nio.charset`

The locale data (names of months, formats, sorting rules) comes from the **Unicode CLDR** project, the default source since Java 9. Java 21 supports Unicode 15.0.

Doing It Right

An internationalization process



- **Pseudo-localization**: replace every text with a longer, accented version (`Save` → `[Šàvé !!!]`) to find hard-coded strings and too-small buttons *before* translation.
- Leave room: German and Finnish texts are often 30% longer than English.
- Test with a Turkish locale: it finds case-mapping and decimal-comma bugs quickly.

Summary

Big ideas

- i18n separates **what varies by culture** from the code; l10n fills it in for one locale.
- A **locale** (`tr-TR`) chooses language and conventions; the default comes from the operating system, so pass a locale when the output must not change.
- Text is **Unicode**; bytes need an **encoding**, preferably UTF-8, at every border.
- Numbers, money, dates, sorting and case all depend on the locale: use the Java classes, not hand-made patterns.
- Translated texts live in **resource bundles**, with placeholders, never glued with +.

Check yourself

1. Why is `i18n` written with 18?
2. What do the parts of `pt-BR` and `sr-Latn-RS` mean?
3. A file shows `BaÅŸlÄ±k`. What went wrong, and what was the original word?
4. Why can `"1.234"` mean two different numbers?
5. Which locale should `toLowerCase` use for an HTTP header name? Why?
6. Why does `Collections.sort` put `Ömer` after `zil`?
7. Why is `"Ali sent " + n + " files"` bad for translation?
8. Where do you change the regional format on your own computer?

Questions?